

TFG Desarrollo de Aplicaciones Multiplataforma



Raúl Mateo Azparren

25/03/2025

2.º DAM

Tutor: Jorge Benito Daniel

Índice

Índice.....	1
Introducción.....	3
Objetivos.....	4
Fases del Proyecto.....	5
Diseño.....	5
Casos de uso.....	5
Arquitectura del Sistema.....	7
Base de Datos.....	9
Tecnologías Usadas.....	11
Angular.....	11
Git.....	11
GitHub.....	11
Draw.io.....	11
Visual Studio Code.....	11
Visual Studio.....	11
TypeScript.....	12
C#.....	12
Docker.....	12
Implementación.....	13
API-Diseñar Base de Datos.....	14
API - Crear los roles de los usuarios.....	15
API - Endpoints Usuarios.....	15
Aplicación web - Inicio Sesión/Registro.....	16
11	
API- Endpoints Cursos.....	17
12	
Aplicación web - Mostrar cursos.....	18
Aplicación web - Detalles de los cursos.....	20
Aplicación web - Inscribirse a los cursos.....	20
Aplicación web - Mostrar tus cursos.....	20
Aplicación web - Crear curso.....	22
Aplicación web - Mostrar estudiantes.....	22
Aplicación web - Borrar Curso.....	23

Aplicación Escritorio - Inicio de sesión.....	24
Aplicación Escritorio - Mostrar Cursos.....	24
Aplicación Escritorio - Mostrar estudiantes.....	25
Aplicación Escritorio - Borrar Curso/ Estudiante.....	25
Aplicación Escritorio - Mostrar usuarios.....	26
Aplicación Escritorio - Borrar usuario.....	27
Pruebas.....	28
Instalación del Proyecto.....	28
Validaciones.....	29
Ampliaciones y posibles mejoras.....	32
Conclusión.....	33
Bibliografía.....	34
Angular.....	34
Docker.....	34
Day.js.....	34
TypeScript.....	34
WPF.....	34
Referencias.....	35

Introducción

DigiClass es una aplicación web que permite a los estudiantes apuntarse a los cursos que proporcionamos de forma gratuita y sencilla.

Además de proporcionar a profesores autónomos que puedan registrarse en nuestra página web y puedan impartir en nuestras clases sus cursos.

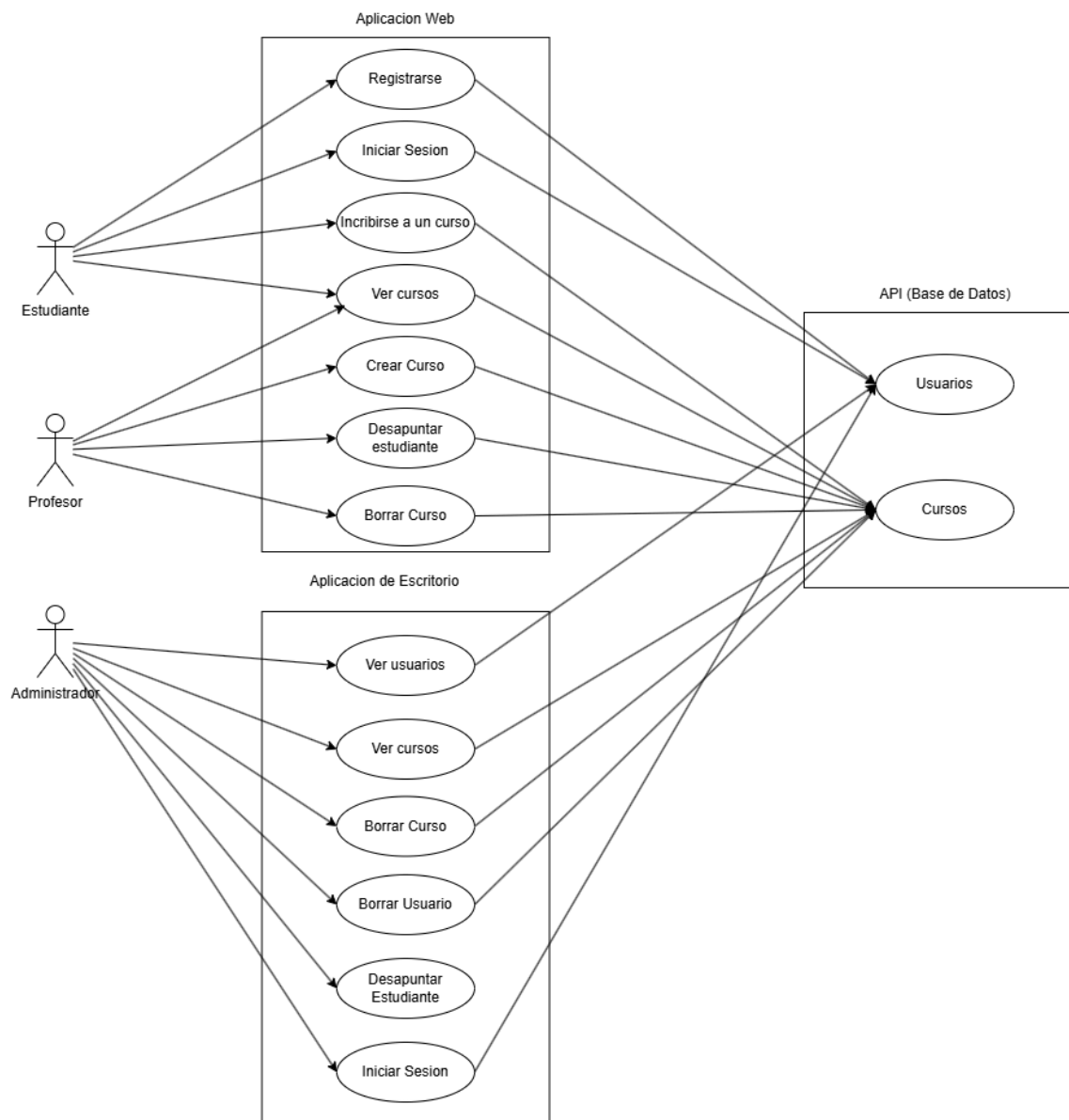
Objetivos

- **Permitir que los estudiantes puedan apuntarse a cursos:** A través de nuestra página web permitiremos que los estudiantes se puedan apuntar a los cursos que proporcionamos.
- **Permitir que los profesores puedan crear cursos:** A través de nuestra página web permitiremos que los profesores puedan crear de manera online cursos.
- **Permitir que el usuario administrador pueda gestionar la aplicación:** Desarrollar un rol que nos permita gestionar tanto los usuarios de la aplicación como los cursos que hay disponibles.
- **Desarrollar una aplicación de escritorio para el usuario administrador:** Con el fin de facilitar la gestión del administrador, desarrollaremos una aplicación de WPF que permite visualizar y eliminar tanto a los usuarios como los cursos creados
- **Desarrollar una API(Application Programming Interface):** Implementaremos una API que nos permita tanto la aplicación web como la aplicación de escritorio conectarnos a la base de datos de DigiClass

Fases del Proyecto

Diseño

Casos de uso



Los 3 actores que acceden a la aplicación web y a la aplicación de escritorios son:

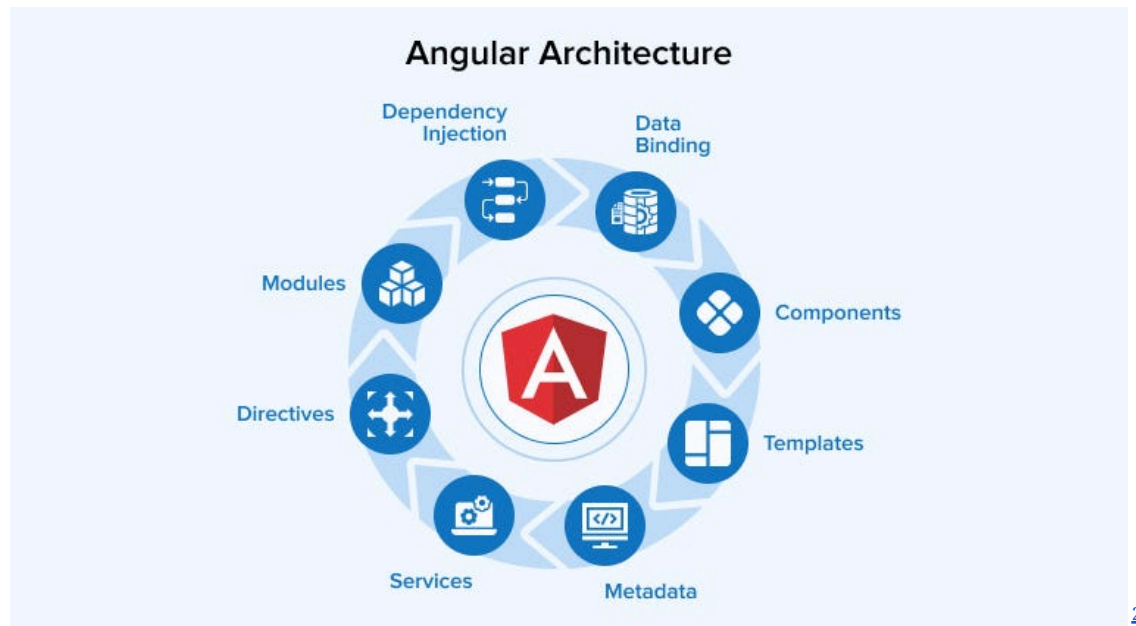
- **Estudiante**
- **Profesor**
- **Administrador**

Cada actor desarrolla un caso de uso:

- Registrarse:
 - Estudiante -> Como estudiante podrá registrarse en nuestra página web.
 - Profesor -> Como profesor podrá registrarse en nuestra página web
- Iniciar Sesión
 - Estudiante, Profesor -> Podrá iniciar sesión en la página web
 - Administrador -> Podrá iniciar sesión en la aplicación de escritorio
- Inscribirse en un Curso
 - Estudiante -> Como estudiante podrá inscribirse en nuestros cursos
- Ver cursos
 - Estudiante, Profesor -> Podrán ver los cursos disponibles en la página web
 - Administrador -> Como administrador podrá ver todos los cursos en la aplicación de escritorio.
- Crear Curso
 - Profesor -> Como profesor podrá crear cursos a través de nuestra página web
- Desapuntar Estudiante
 - Profesor -> Como profesor podrá desapuntar a los estudiantes que se hayan inscrito en sus cursos
 - Administrador -> Como administrador podrá despuntar a cualquier estudiante en cualquier curso.
- Borrar Curso
 - Profesor -> Como profesor podrá borrar cualquier curso que haya creado
 - Administrador -> Como administrador podrá borrar cualquier curso
- Ver usuarios
 - Administrador -> Como administrador podrá ver todos los usuarios de la base de datos.
- Borrar usuario
 - Administrador -> Como administrador podrá borrar cualquier usuario de la base de datos

Arquitectura del Sistema

La arquitectura para la aplicación web sigue una arquitectura orientada a **componentes** que es la que usa **Angular**, el framework que utilice para desarrollar la aplicación.



Cada componente hace referencia a una interfaz de usuario, ya sea un botón o una página web.

Además cada componente tiene asociado:

- Un archivo typeScript que contiene la lógica
- Un archivo html que contiene la vista
- Un archivo css que contiene la definición del diseño del componente

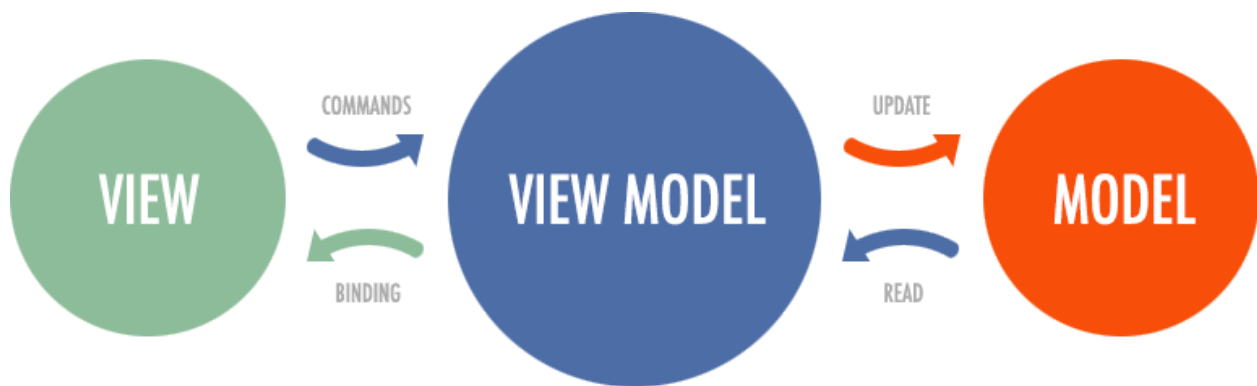
component	02/05/2025 18:03	Carpeta de archivos	
models	01/05/2025 18:56	Carpeta de archivos	
pages	27/04/2025 16:38	Carpeta de archivos	
service	27/04/2025 17:44	Carpeta de archivos	
app.component.css	26/04/2025 18:28	Documento de ho...	2 KB
app.component.html	27/04/2025 15:06	Microsoft Edge H...	1 KB
app.component.ts	02/05/2025 18:11	Archivo TS	2 KB
app.config.ts	06/03/2025 17:21	Archivo TS	1 KB
routes.ts	27/04/2025 17:44	Archivo TS	2 KB

3

Para la aplicación de escritorio use la tecnología **WPF(Windows Presentation Foundation)** siguiendo una arquitectura **MVVM (Model-View-ViewModel)** que es el patrón de diseño recomendado para separar la lógica de presentación de la interfaz de usuario.

Este patrón de arquitectura está dividido en:

- **Model (Modelo):** Representa los datos y la lógica de esos datos
- **ViewModel (Vista de Modelo):** Actúa como intermediario entre la vista y el modelo, y contiene la lógica que prepara los datos para la vista
- **View (Vista):** Es la interfaz que ve el usuario



4

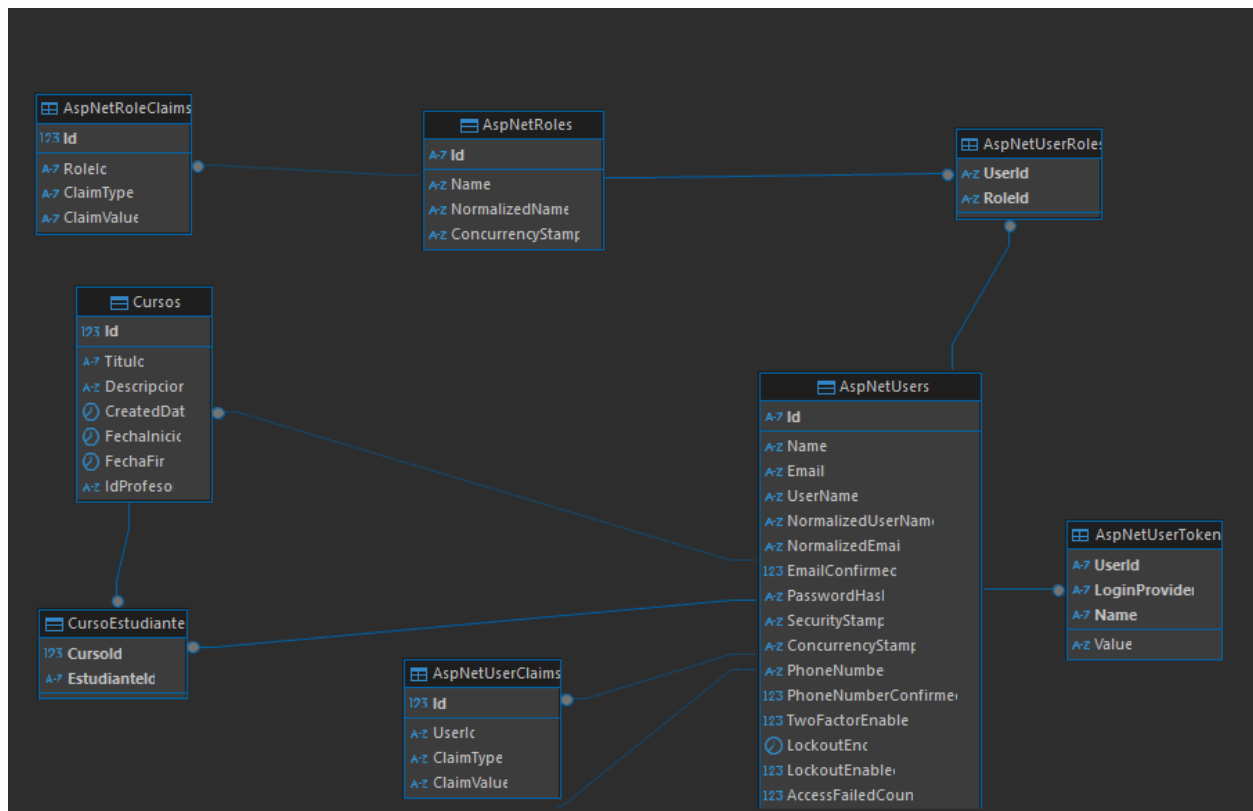
Para que tanto la aplicación web como la aplicación de escritorio se puedan comunicar con la base de datos diseñe una **API(Application Programming Interface)**, que es un conjunto de reglas que permite que diferentes programas o sistemas se comuniquen entre sí.

Junto con la API diseñe varios **endpoints**, que representan un recurso o una acción.

- **Cursos**
 - GET -> Devuelve todos los cursos de la base de datos
 - POST -> Crea un curso y lo guarda en la base de datos
 - /Nuevos GET -> Devuelve los cursos en los que el estudiante no está inscrito
 - /{id} GET -> Devuelve el curso con el id indicado
 - /{id} PUT -> Modifica el curso con el id indicado
 - /{id} DELETE -> Elimina el curso con el id indicado
 - /Participante/{cursoId}&&{userId} DELETE -> Elimina la inscripción del estudiante con el id indicado en el curso indicado

- User
 - GET -> Devuelve todos los usuarios
 - /MisCursos GET -> Devuelve todos los cursos en el que estudiante esté inscrito
 - /Participantes/{cursoId} GET -> Devuelve todos los estudiantes inscritos en el curso indicado
 - /Apuntarse/{cursoId} POST -> Inscribe al estudiante en el curso indicado
 - DELETE -> Elimina al usuario
 - /register POST -> Registra al usuario en la base de datos
 - /login POST -> Inicia sesión con el correo y contraseña proporcionado

Base de Datos



La base de datos está compuesta principalmente de 2 tablas `AspNetUser`, que es la tabla con todos los usuarios, y `Cursos`, que es la tabla con todos los cursos.

También hay otras tablas que usa el propio sistema para el funcionamiento interno, como la de las migraciones y otras tablas para la relación de roles con los usuarios.

Tecnologías Usadas

Angular

Angular es un framework para aplicaciones web desarrollado en TypeScript, de código abierto, mantenido por Google, que se utiliza para crear y mantener aplicaciones web de una sola página.

Es el framework que usé para el desarrollo de mi aplicación web.

Git

Git es un sistema de control de versiones que realiza un seguimiento de los cambios en los archivos. Es muy usado en los trabajos donde varias personas tienen que modificar los mismos archivos.

GitHub

GitHub es un servicio basado en la nube que aloja un sistema de control de versiones llamado Git. Éste permite a los desarrolladores colaborar y realizar cambios en proyectos compartidos, a la vez que mantienen un seguimiento detallado de su progreso.

Draw.io

Draw.io es una herramienta de diagramación online, gratuita y de código abierto, que te permite crear diferentes tipos de diagramas, como diagramas de flujo, UML, mapas conceptuales, entre otros.

Visual Studio Code

Visual Studio Code es un editor de código fuente desarrollado por Microsoft para Windows, Linux, macOS y Web.

Visual Studio

Visual Studio es un Entorno de Desarrollo Integrado (IDE) de Microsoft, utilizado por

desarrolladores para crear aplicaciones de software. Es una herramienta completa que permite editar, depurar, compilar y publicar código en diferentes plataformas

TypeScript

TypeScript es un lenguaje de programación de tipado estático que se compila a JavaScript. En esencia, es un superconjunto de JavaScript que añade características como tipos estáticos, clases e interfaces para mejorar la escalabilidad y mantenibilidad del código.

C#

Es un lenguaje de programación de propósito general, multiplataforma y orientado a objetos, desarrollado por Microsoft. Es el lenguaje de programación más popular para el desarrollo .NET, que se utiliza para crear una amplia gama de aplicaciones.

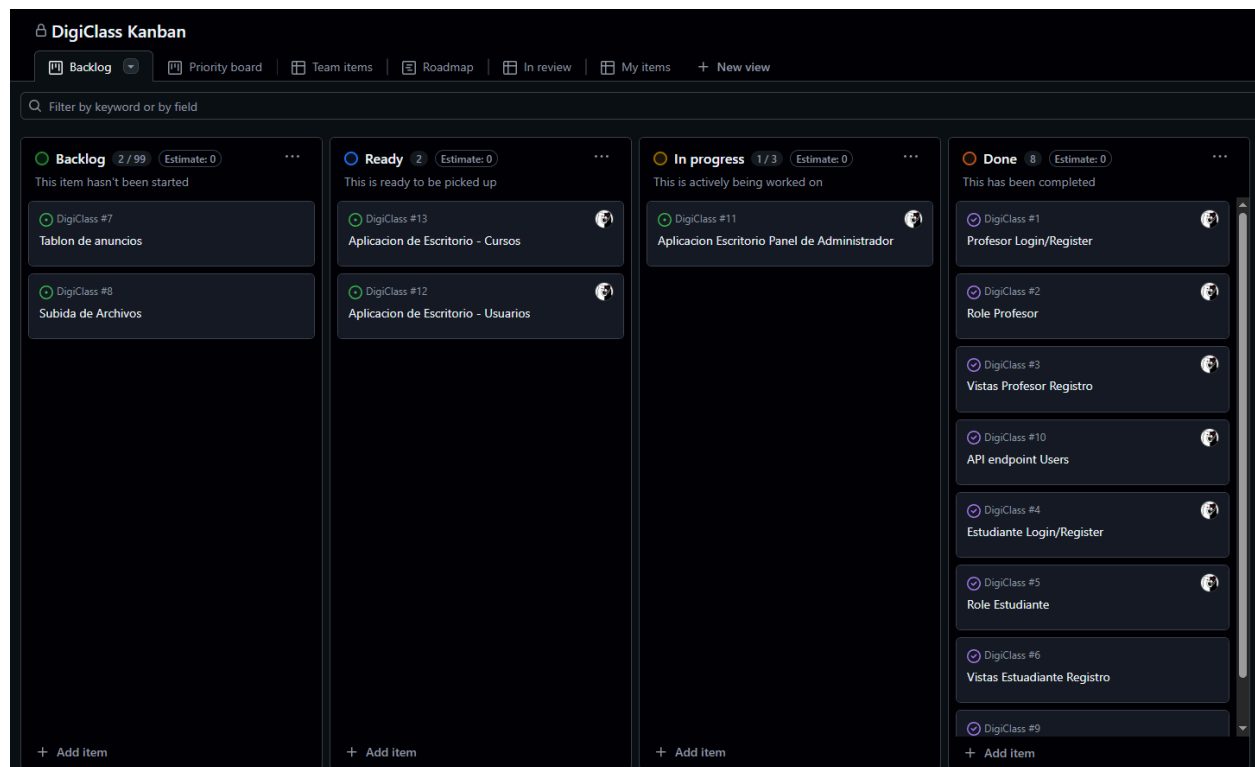
Docker

Docker es una plataforma de código abierto que permite empaquetar, enviar y ejecutar aplicaciones en contenedores. Los contenedores son entornos de ejecución aislados que incluyen el código de la aplicación, las librerías, las herramientas y las dependencias necesarias para que funcione.

Implementación

Para la organización del proyecto seguí la metodología ágil Kanban, e hice un tablero de Kanban con GitHub con todas las tareas del proyecto.

Fui asignando cada una de las tareas en función de la prioridad e importancia que suponía la tarea.



6

Estas son cada una de las tareas del Kanban que fui realizando:

API-Diseñar Base de Datos

Para el diseño de la base de datos inicie un proyecto [ASP.NET](#) Core, que me facilitaba una plantilla para la realización de la base de datos, con lo que solo tuve que realizar cada uno de las entidades(Cursos, Users) indicando en cada caso su relación (1:N, N:N)

```

20 referencias
public class AppUser : IdentityUser
{
    4 referencias
    public string Name { get; set; }
    2 referencias
    public string Email { get; set; }

    3 referencias
    public List<CursoEntity> CursosProfesor { get; set; }
    6 referencias
    public List<CursoEntity> Cursos { get; set; }
}

```

2

```

public class CursoEntity
{
    [Key]
    [DatabaseGenerated(DatabaseGeneratedOption.Identity)]
    7 referencias
    public int Id { get; set; }

    [Required]
    [MaxLength(50)]
    0 referencias
    public string Titulo { get; set; }

    0 referencias
    public string Descripcion { get; set; }
    2 referencias
    public DateTime CreatedDate { get; set; }

    [Required]
    2 referencias
    public DateTime FechaInicio { get; set; }

    [Required]
    0 referencias
    public DateTime FechaFin { get; set; }
    [Required]
    2 referencias
    public string IdProfesor { get; set; }

    6 referencias
    public AppUser Profesor { get; set; }

    6 referencias
    public List<AppUser> Participantes { get; set; }
}

```

API - Crear los roles de los usuarios

Para el control de usuarios y permisos tanto de la aplicación web como la de escritorio tuve que crear 3 roles (Estudiante, Profesor, Administrador).

Al usar la plantilla que tiene [ASP.NET](#), me proporcionaba de forma muy sencilla crear los roles de los usuarios.

```
{  
    //esto solo se ejecutara la primera vez que los roles sean creados  
    await _roleManager.CreateAsync(new IdentityRole("admin"));  
    await _roleManager.CreateAsync(new IdentityRole("estudiante"));  
    await _roleManager.CreateAsync(new IdentityRole("profesor"));  
}
```

API - Endpoints Usuarios

Para el funcionamiento correcto de las aplicaciones tuve que desarrollar los endpoints de los Usuarios con las distintas acciones(borrar usuario, registrarse, iniciar sesión) que requiere para que las aplicaciones funcionen correctamente.


```
0 referencias
public async Task<IActionResult> Register(UserRegistrationDto userRegistrationDto)
{
    if (!ModelState.IsValid)
    {
        return BadRequest(new { error = "Incorrect Input", message = ModelState });
    }

    if (!_userRepository.IsUniqueUser(userRegistrationDto.UserName))
    {
        _reponseApi.StatusCode = HttpStatusCode.BadRequest;
        _reponseApi.IsSuccess = false;
        _reponseApi.ErrorMessages.Add("Username already exists");
        return BadRequest();
    }

    var newUser = await _userRepository.Register(userRegistrationDto);
    if (newUser == null)
    {
        _reponseApi.StatusCode = HttpStatusCode.BadRequest;
        _reponseApi.IsSuccess = false;
        _reponseApi.ErrorMessages.Add("Error registering the user");
        return BadRequest();
    }

    _reponseApi.StatusCode = HttpStatusCode.OK;
    _reponseApi.IsSuccess = true;
    return Ok(_reponseApi);
}
```

Aplicación web - Inicio Sesión/Registro

Para que los usuarios de la aplicación web pudieran registrarse e iniciar sesión en la página web cree las vistas para que los usuarios pudieran interactuar.

DigiClass Inicio Cursos Sobre Nosotros Contacto

Iniciar Sesión **Registrarse**

Inicia Sesión

Correo electrónico

Contraseña

Entrar

10

Además de crear el servicio en angular para mandar los datos del usuario para iniciar sesión o registrarse a la API.

```

async login(credentials: LoginDTO): Promise<Result> {
    const responseFetch=await fetch(this.loginUrl, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(credentials)
    })
    .then(response => response.json())
    .catch(error => {
        throw new Error(error)
    });

    //console.log('Login response:', responseFetch);
    if (responseFetch?.result?.token) {
        const response:Result={
            token: responseFetch.result.token,
            name:responseFetch.result.name,
            role:responseFetch.result.role
        }
        return response
    } else {
        throw new Error('Token no valido')
    }
}

```

11

API- Endpoints Cursos

Para que los usuarios puedan crear sus cursos, verlos e inscribirse tuve que desarrollar los endpoints de los cursos con las diferentes acciones.

```

[Route("api/[controller]")]
[ApiController]
1 referencia
public class CursoController : ControllerBase
{
    private readonly ICursoRepository _cursoRepository;
    private readonly IMapper _mapper;
    //protected readonly ILogger _logger;

    0 referencias
    public CursoController(ICursoRepository repository, IMapper mapper /*ILogger logger*/)
    {
        _cursoRepository = repository;
        _mapper = mapper;
        //_logger = logger;
    }

    [HttpGet]
    //[Authorize(Roles = "profesor,estudiante")]
    [AllowAnonymous]
    [ProducesResponseType(StatusCodes.Status200OK)]
    0 referencias
    public async Task<IActionResult> GetAll()
    {
        try
        {
            var entities = _mapper.Map<List<CursoDTO>>(await _cursoRepository.GetAllAsync());

```

12

Aplicación web - Mostrar cursos

Para que los usuarios pudieran ver los cursos que había disponible creé un servicio que hacía una petición a la API para que me sacara todos los cursos que hay en la base de datos.

Después les muestro como componentes en la página web.

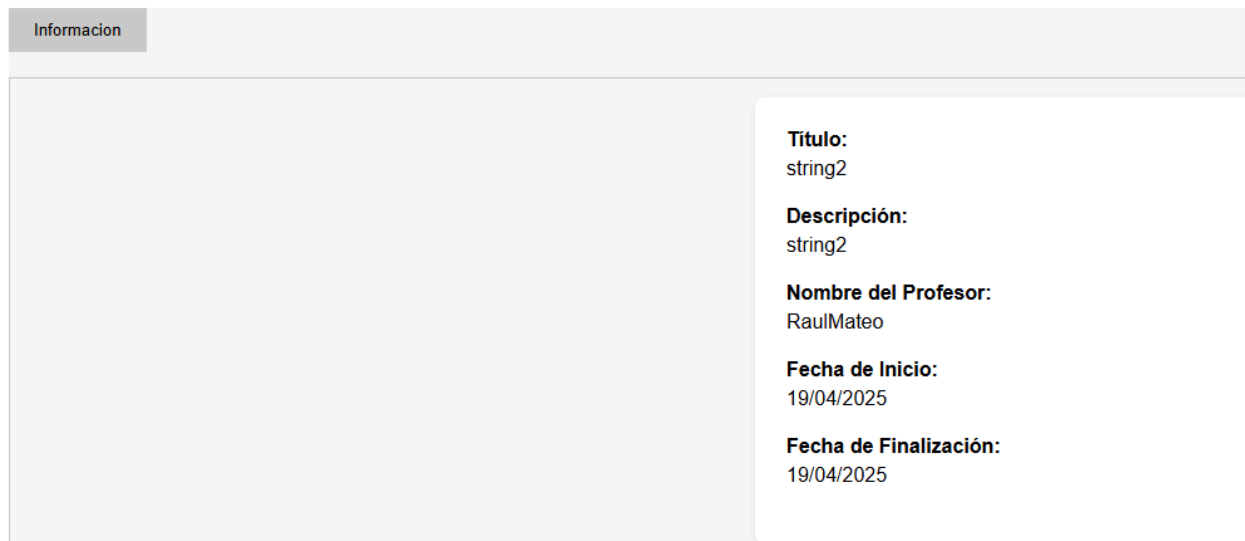
```
@Component({
  selector: 'app-cursos',
  imports: [CommonModule, CardCursoComponent],
  templateUrl: './cursos.component.html',
  styleUrls: ['./cursos.component.css']
})
export class CursosComponent {
  cursosList: Curso[] = [];

  constructor(private cursoService: CursoService) {
    if(localStorage.getItem('token')){
      cursoService.CursosLogin().then((cursos)=>{
        this.cursosList=cursos
      })
    }else{
      cursoService.Cursos().then((cursos)=>{
        this.cursosList=cursos
      })
    }
  }
}
```

[13](#)

Aplicación web - Detalles de los cursos

Para que los estudiantes pudieran ver los detalles(Profesor, Fecha de inicio,etc) hice otro componente que me mostrara la información del curso seleccionado



Informacion

Título:
string2

Descripción:
string2

Nombre del Profesor:
RaulMateo

Fecha de Inicio:
19/04/2025

Fecha de Finalización:
19/04/2025

[14](#)

Aplicación web - Inscribirse a los cursos

Con el fin de que los estudiantes pudieran inscribirse a los cursos, añadí un botón debajo de los detalles del curso en que los estudiantes pudieran inscribirse de forma rápida y sencilla al curso en cuestión.

Fecha de Finalización:
15/04/2025

Inscribirse

[15](#)

Aplicación web - Mostrar tus cursos

```
async InscribirseCurso(Id: number): Promise<boolean> {
  this.token = localStorage.getItem('token');

  const response=await fetch(this.inscribirse+'/${Id}', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json','Authorization': `Bearer ${this.token}` },
  }).catch(error => {
    throw new Error(error)
  });

  if (!response.ok) {
    throw new Error('Algo salio mal')
  }else{
    return true
  }
}
```

[16](#)

Con el fin de que los usuarios pudieran ver los cursos en los que estaban inscritos, o en caso de ser profesor, los que había creado, cree una nueva página que te mostraba tus cursos en caso de haber iniciado sesión, para ello manda una petición a la API pidiendo los cursos.

```
async MisCursos(): Promise<[Curso]> {
  this.token = localStorage.getItem('token');
  const response=await fetch(this.misCursos, {
    method: 'GET',
    headers: { 'Content-Type': 'application/json','Authorization': `Bearer ${this.token}` },
  }).then((data)=>data.json())
  .catch(error => {
    throw new Error(error)
  });
  return response;
}
```

[17](#)

Aplicación web - Crear curso

Para que los profesores puedan crear cursos, agregue un botón en el cual muestra una ventana con los campos a rellenar para crear un curso.



Nuevo Curso

Título

Descripción

Fecha de inicio

Fecha final

Cancelar Guardar

[18](#)

Aplicación web - Mostrar estudiantes

Con el fin de que los profesores pudieran ver los estudiantes que están inscritos en su curso y poder eliminarlos, implemente una pestaña que les mostraba todos los estudiantes inscritos en el curso haciendo una petición a la API, además de incluir un botón que borraba la inscripción del estudiante que el profesor seleccione.

Informacion Participantes		
Nombre	Correo	Acciones
mateo	ra@gmail.com	Eliminar

[19](#)

Aplicación web - Borrar Curso

Con el fin de que los profesores puedan borrar los cursos que hayan creado, añadí un botón en los detalles del curso de forma que el profesor que haya creado el curso pueda eliminar el curso al pulsarlo.



[20](#)

Aplicación Escritorio - Inicio de sesión

Con el fin de que solo los administradores puedan entrar a la aplicación implemente un inicio de sesión que me identifica el rol del usuario y si posee el rol de administrador le deja acceder a la aplicación.



Correo Electronico

Contraseña

Enviar

21

Aplicación Escritorio - Mostrar Cursos

Con el fin de que los administradores pudieran ver todos los cursos de la base de datos, desarrollé una vista que me mostraba los datos de todos los cursos de la base de datos en forma de una tabla.



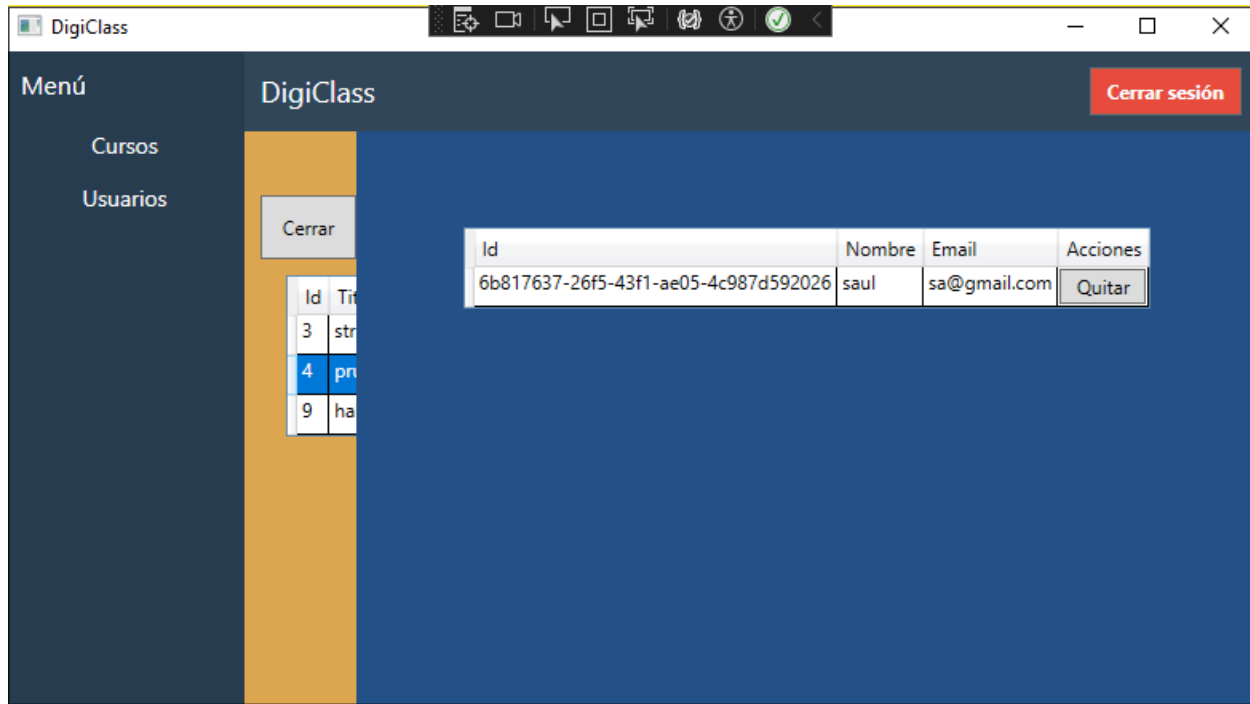
Id	Titulo	Descripcion	Profesor	Fecha de Inicio	Fecha de fin	Ver	Acciones
3	stringRes	string	RaulMateo	4/26/2025 2:41:41 PM	4/26/2025 2:41:41 PM	Ver	Eliminar
4	prueba	string	RaulMateo	4/26/2025 2:47:37 PM	4/26/2025 2:47:37 PM	Ver	Eliminar
9	hala	affsf	RaulMateo	5/22/2025 12:00:00 AM	5/24/2025 12:00:00 AM	Ver	Eliminar

Atras 1 Siguiente

22

Aplicación Escritorio - Mostrar estudiantes

Para que los administradores puedan ver los estudiantes que hay apuntados al curso añadí un botón que me muestra un desplegable con los estudiantes apuntados al curso.



23

Aplicación Escritorio - Borrar Curso/ Estudiante

Con el fin de que los administradores puedan borrar los cursos de la base de datos añadí un botón para eliminar el curso que seleccionen, también añadí un para eliminar al estudiante seleccionado del curso.

```

2 referencias
public async Task<bool> Delete(int cursoId, string token)
{
    using HttpClient httpClient = new HttpClient();
    {
        httpClient.DefaultRequestHeaders.Add("Authorization", $"Bearer {token}");
        HttpResponseMessage datos = await httpClient.DeleteAsync(Constants.CURSOS + "/" + cursoId.ToString());
        string dataget = await datos.Content.ReadAsStringAsync();
        //return JsonSerializer.Deserialize<T>(dataget);
        if (datos.IsSuccessStatusCode)
        {
            return true;
        }
        else
        {
            return false;
        }
    }
}

```

24

Aplicación Escritorio - Mostrar usuarios

Con el fin de que los administradores puedan ver todos los usuarios de la base de datos desarrolle una una vista que me mostraba los datos de todos los usuarios de la base de datos en forma de una tabla.

The screenshot shows the DigiClass desktop application interface. On the left is a dark blue sidebar menu with options: 'Cursos' and 'Usuarios'. The main area has a header 'DigiClass' with a 'Cerrar sesión' button. Below the header is a search section with a 'Filtro' label, a 'Recargar' button, a search input field, and a 'Buscar' button. The main content is a table with 5 columns: 'Id', 'Nombre', 'Correo', 'Rol', and 'Acciones'. The table contains 6 rows of user data. At the bottom, there are pagination controls: 'Atras', '1', and 'Siguiente'.

Id	Nombre	Correo	Rol	Acciones
36aadf14-861b-490e-964c-21fa166b9ca7	mateo20	rafa@gmail.com	profesor	Eliminar
52eb1a0a-6536-4ede-a7ae-4a8e5b427137	mateoRaul	rfa@gmail.com	estudiante	Eliminar
6351fe31-4a01-40ad-9b70-94775137a0f2	Raul	rma@gamil.com	estudiante	Eliminar
5c5b8714-c664-4184-884a-a1215a38b54b	RaulMateo	rasa@gmail.com	profesor	Eliminar
f47344a0-6d2c-4d66-9510-8fb6b4e0fd8a	sago	sago@gmail.com	estudiante	Eliminar

25

Aplicación Escritorio - Borrar usuario

Con el fin de que los administradores puedan borrar los usuarios de la base de datos añadí un botón para eliminar el usuario que seleccionen.

```
[RelayCommand]
2 referencias
private async void Delete(UsersModel user)
{
    if (user != null)
    {
        MainViewModel mainWindow = App.Current.Services.GetService<MainViewModel>();

        if(await _productService.Delete(user.id, mainWindow.GetToken()))
            Items.Remove(user);
    }
}
```

[26](#)

Pruebas

Instalación del Proyecto

Para la instalación del proyecto necesitaremos tener instalado:

- Node
- Visual Studio con los paquetes de desarrollo de [ASP.NET](#) y desarrollo de escritorio .NET
- Docker con un contenedor de Microsoft SQL Server o directamente una base de datos de Microsoft SQL Server

Para el despliegue de la API abriremos el proyecto `RestAPI` y cambiaremos el archivo `appsettings.json` para que nos conecte a nuestra base de datos.

```
{
  "ApiSettings": {
    "SecretKey": "uE/!Wx9<t2{V73@reb^4]6c)N#TqgA;s&XHP_nf}d%+S>*"
  },
  "claveSecreta": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",
  "ConnectionStrings": {
    "SqlConnection": "Server=localhost;Database=DigiClass;User=AppUserLogin;Password=e.d_fwm2()~37hz?+LBT4V;TrustServerCertificate=True"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning",
      "MatchMaker.API": "Debug"
    }
  }
}
```

[27](#)

Una vez cambiado, abriremos la consola del administrador de paquetes y escribiremos `Update-Database`, con ello se nos aplicaran todas las migraciones necesarias a la base de datos, lo que nos añadirá las tablas necesarias. Una vez terminado ya podremos ejecutar la API

Para el uso de la aplicación de escritorio bastará con que abras el proyecto `DigiClass` con visual studio y ejecutes la aplicación.

Para el uso de la aplicación web tendremos que abrir una consola dentro de la carpeta `AngularFront`, una vez situado ahí escribiremos los siguientes comando

1. `npm install`
2. `npm run build`

3. `ng serve`

Con el 3º comando se nos desplegará la aplicación web en local.

Validaciones

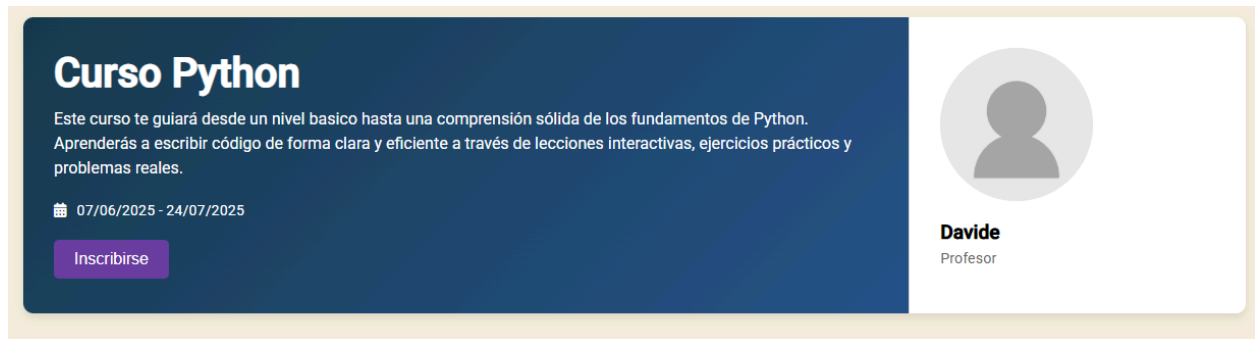
Probé a crear a 2 usuarios a través de la aplicación web, un profesor y un estudiante, una vez creados comprobé que aparecían los registros en la base de datos con los roles correspondientes.

28

AspNetUsers Enter a SQL expression to filter results (use Ctrl+Space)					
	A-Z Id	A-Z Name	A-Z Email	A-Z UserName	A-Z NormalizedUserName
1	7c1e10ac-a1ec-483e-8781-166549b9d6f7	Raul	ra@gmail.com	Raul	RAUL
2	c32439bb-3119-498b-8616-1510054d1915	Davide	da@gmail.com	Davide	DAVIDE

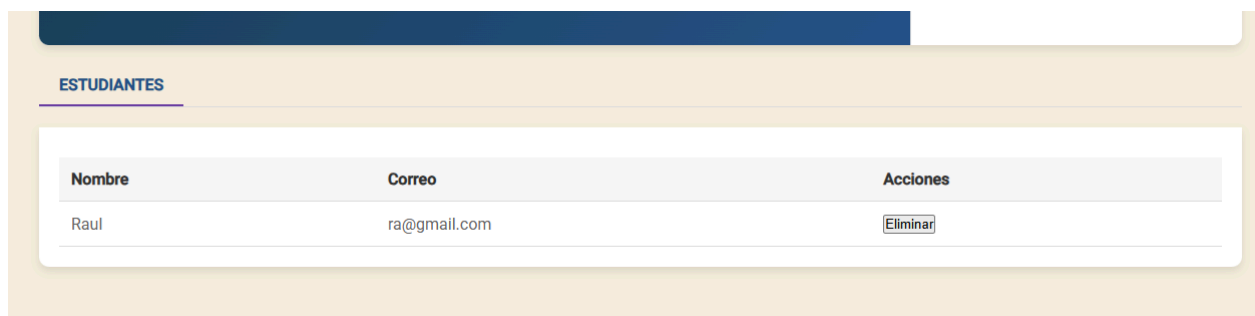
29

Luego probé a crear un curso con la cuenta del profesor e inscribirse a él con la cuenta de estudiante.



[30](#)

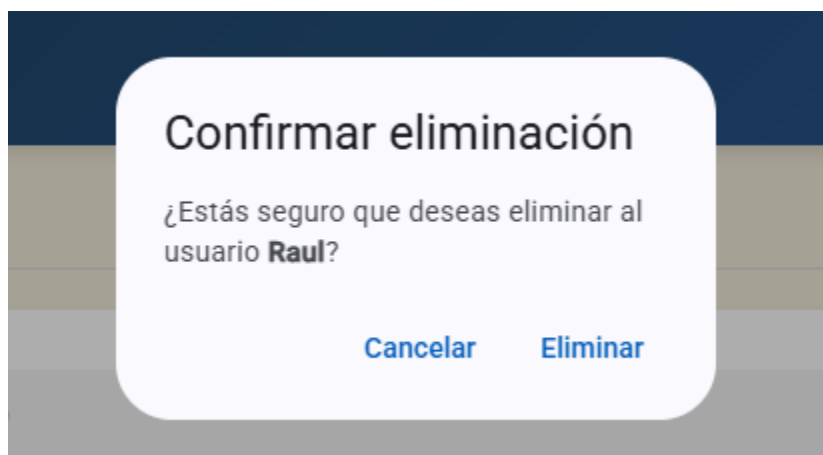
Una vez inscrito, comprobé que el estudiante salía reflejado como estudiante en el curso.



Nombre	Correo	Acciones
Raul	ra@gmail.com	<button>Eliminar</button>

[31](#)

Después probé a eliminar al estudiante del curso y comprobar que no volvía a aparecer, también probé a eliminar el curso y comprobando que no aparecía en la base de datos.



Confirmar eliminación

¿Estás seguro que deseas eliminar al usuario **Raul**?

Cancelar Eliminar

[32](#)

Probé a entrar como administrador en la aplicación de escritorio y eliminar tanto un curso como un usuario.

The screenshot shows the DigiClass application interface. On the left is a dark blue sidebar menu with the following items: "Menú", "Cursos", and "Usuarios". The main content area has a dark blue header with "DigiClass" and a red "Cerrar sesión" button. Below the header is a light orange section with a "Filtro" label, a "Recargar" button, a search input field, and a "Buscar" button. A table displays a list of courses:

Id	Titulo	Descripción	Profesor	Fecha de Inicio	Fecha de fin	Ver	Acciones
2	Python	Cursoooo	Davide	6/4/2025 12:00:00 AM	6/6/2025 12:00:00 AM	Ver	Eliminar

At the bottom of the table area, there are pagination controls: "Atras 1 Siguiete".

33

Ampliaciones y posibles mejoras

- **Subir imágenes:** Me habría gustado poder implementar un servicio que te dejara subir imágenes al servidor y que los usuarios pudieran poner imágenes en los cursos o fotos de perfiles, pero por falta de recursos (nube) tuve que descartar.
- **Subida de archivos:** Me habría gustado implementar la opción de que los profesores pudieran subir archivos a los cursos que hayan creado y así ampliar el contenido que ofrecen, pero por la misma razón que no puede implementar lo de subir imágenes lo tuve que descartar.
- **Filtrar los cursos:** Me habría gustado añadir un filtro de categorías a la hora de buscar cursos para facilitar la búsqueda a los estudiantes, aunque por falta de tiempo no puede añadirlo.
- **Clases online:** En un principio quise añadir la opción de que las clases fueran online y que los profesores pudieran iniciar una videollamada con los estudiantes, pero debido a varios errores que me daba y la complejidad de ello tuve que descartar.

Conclusión

El desarrollo de DigiClass ha permitido poner en práctica los conocimientos adquiridos a lo largo del grado, integrando diferentes tecnologías modernas para intentar dar solución a una necesidad real: la creación de una plataforma web que facilite la gestión y participación en cursos.

A lo largo del proyecto, se ha implementado un sistema completo compuesto por una interfaz de usuario dinámica y responsiva desarrollada en Angular, junto a una API construida en ASP.NET para gestionar la lógica de negocio, autenticación, almacenamiento y comunicación con la base de datos. Esta arquitectura junto a Docker ha permitido lograr una separación clara entre el frontend y el backend.

DigiClass, en su versión actual, ofrece funcionalidades básicas como la creación y gestión de cursos, inscripción de usuarios y visualización de contenidos. Sin embargo, el proyecto tiene mucho margen de mejora, como la incorporación de videollamadas en tiempo real, integraciones con plataformas externas de contenido educativo.

En resumen, este trabajo ha sido una oportunidad valiosa para mejorar mis habilidades y metodológicas en un entorno de desarrollo real, usando varias tecnologías modernas que se usan actualmente.

Bibliografía

Angular

- Documentación Angular: [Angular](#)
- Routing : <https://angular.dev/guide/routing>
- Componentes: <https://angular.dev/guide/components>
- Video : <https://youtu.be/f7unUpshmpA?si=VicDP3cEaLG5G1Ia>

Docker

- Documentación: <https://docs.docker.com/>
- Contenedor de MSQl: <https://hub.docker.com/r/microsoft/mssql-server>

Day.js

- Documentación: <https://day.js.org/en/>

TypeScript

- Documentación: <https://www.typescriptlang.org/docs/>

WPF

- Documentación: <https://learn.microsoft.com/es-es/dotnet/desktop/wpf/overview/>

Referencias

1. Diagrama de casos de uso
2. Mapa mental de la arquitectura de Angular
3. Archivos de la aplicación web
4. Mapa mental de la arquitectura MVVM
5. Diagrama de la base de datos
6. Kanban del proyecto
7. Archivo [AppUser.cs](#)
8. Creación de roles en el archivo [UserRepository.cs](#)
9. Código para el registro en el archivo [UserController.cs](#)
10. Página de Inicio de sesión
11. Código del login en el archivo [CursoService.ts](#)
12. Endpoints del archivo CursoController.cs
13. Código para mostrar los curso en el archivo cursos.component.ts
14. Página con los detalles del curso
15. Botón para inscribirse al curso
16. Código para inscribirse a los cursos
17. Código que lanza una petición con los cursos disponibles
18. Formulario para la creación de los cursos
19. Listado de los estudiantes inscritos en el curso
20. Página con los detalles del curso y el botón para borrar el curso
21. Pestaña de inicio de sesión de la aplicación de escritorio
22. Tabla con la lista de los cursos en la aplicación de escritorio
23. Listado de los estudiantes inscritos al curso en la aplicación de escritorio
24. Código que lanza una petición para borrar un curso
25. Tabla con los usuarios en la aplicación de escritorio
26. Código que borra un usuario de la tabla
27. Archivo con la configuración para conectarnos a la base de datos
28. Formulario de registro como estudiante
29. Registros de varios usuarios en la base de datos
30. Detalles de un curso creado en la página web
31. Listado de estudiantes para el curso seleccionado
32. Confirmación para eliminar al usuario
33. Tabla con los cursos después de haber eliminado un curso en la aplicación de

escritorio